How to Create New Validation Rules

1. Identify Your Validation Logic

Look at your existing code and identify discrete validation rules:

```
Coordinate checks
Date/time checks
Species code checks
etc.
```

2. Create Rule Functions

For each type of validation, create a rule function:

```
function your_existing_rule(data, collector, config)
    % PORT YOUR EXISTING LOGIC HERE

    % Example: If you had something like:
    % invalid_lat = data.LAT_DD < 35 | data.LAT_DD > 50;

    % Convert to:
    invalid_idx = find(data.LAT_DD < config.survey_lat_min | ...
                       data.LAT_DD > config.survey_lat_max);

    if ~isempty(invalid_idx)
        collector.addError('LAT_DD', invalid_idx, ...
            'Latitude outside survey area', 'warning');
    end
end
```

3. Register Rules in SurveyValidator

Add your rule to the runValidationRules method:

```
function runValidationRules(obj, data)
    % Existing rules
    narwc.validation.rules.coordinate_rules(data, obj.collector, obj.config);

    % Your ported rule
    narwc.validation.rules.your_custom_rule(data, obj.collector, obj.config);
end
```

4. Write Tests

For each rule you port, write a test:

```
function testYourCustomRule(testCase)
```

```
    data = create_test_data_with_known_issues();
    collector = narwc.validation.ErrorCollector();
    cfg = load_config(); cfg = cfg.validation;  % always pass config
    narwc.validation.rules.your_custom_rule(data, collector, cfg);

    testCase.verifyGreaterThan(collector.getErrorCount(), 0);
end
```

NUMBER and NUMCALF threshold cascade

The `species_rules.m` module flags unusually large group sizes (NUMBER) and calf counts (NUMCALF) using a three-level cascade:

1. SPECCODE-level override — `SPECCODE.typical_max_group` / `typical_max_calf`. Non-NULL value wins over all lower levels. Use for individual species with known atypical group sizes (e.g., a pelagic dolphin species that aggregates in thousands).
2. TAXCODE-level default — `TAXCODE.typical_max_group` / `typical_max_calf`. Applies to all species in that taxonomic group when no SPECCODE override is set.
3. Global fallback — `config.thresholds.group_size_default` (default: 1000) and `config.thresholds.calf_count_default` (default: 100). Applies when both the SPECCODE and TAXCODE columns are NULL.

When a threshold is exceeded, the validator emits a `warning`-level issue with rule IDs `species_rules.number_unusual` or `species_rules.numcalf_unusual`. The warning message includes the threshold value and the source level that produced it, e.g.: `NUMBER=2500 exceeds threshold 2000 for SPECCODE=SADO (source: TAXCODE 3 default)`.

Adjusting thresholds

Thresholds are stored in `data/tables/SPECCODE.csv` (columns `typical_max_group` and `typical_max_calf`) and `data/tables/TAXCODE.csv` (same column names). Edit the CSV, then run `push_lookup_tables.m` to update the database. No code change is needed.

Example: to raise the group-size threshold for Atlantic spotted dolphins (ASDO): 1. Find the ASDO row in `data/tables/SPECCODE.csv`. 2. Set `typical_max_group` to the desired value (e.g., 500). 3. Save the file and run `scripts/setup/push_lookup_tables.m`.

If a SPECCODE row has NULL for `typical_max_group`, the validator uses the TAXCODE-level threshold for that species' taxonomic group. Editing `TAXCODE.csv` changes the default for the entire group at once.